GCWUS

ADVANCED PROGRAMMING
LAB MANUAL

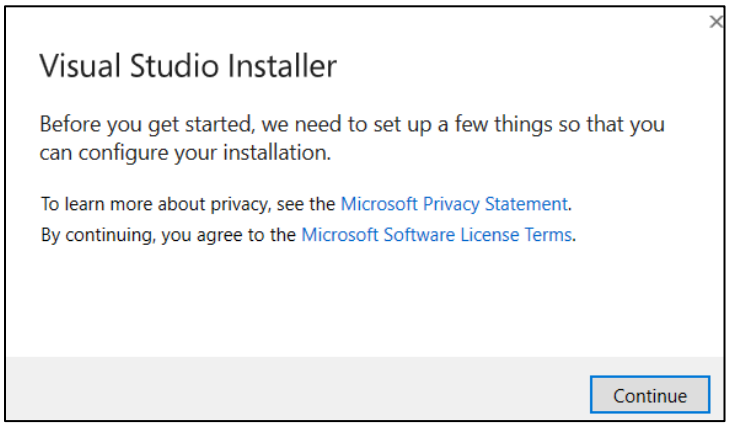
Submitted To: Ma'am Hira

Eman Nisar Ahmad

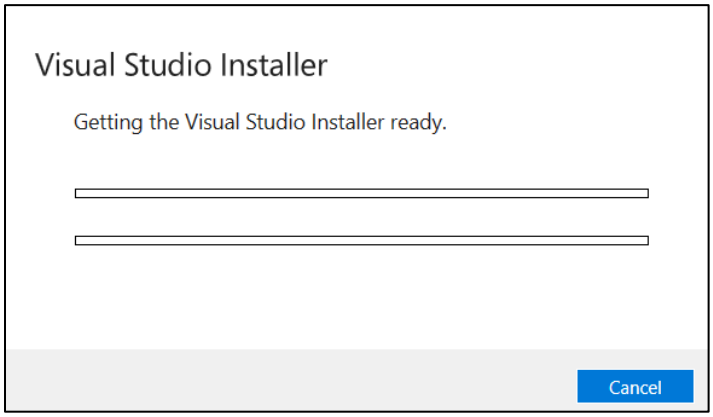
BS-IT-F-23-R-43

LAB 01 (INSTALLATION)

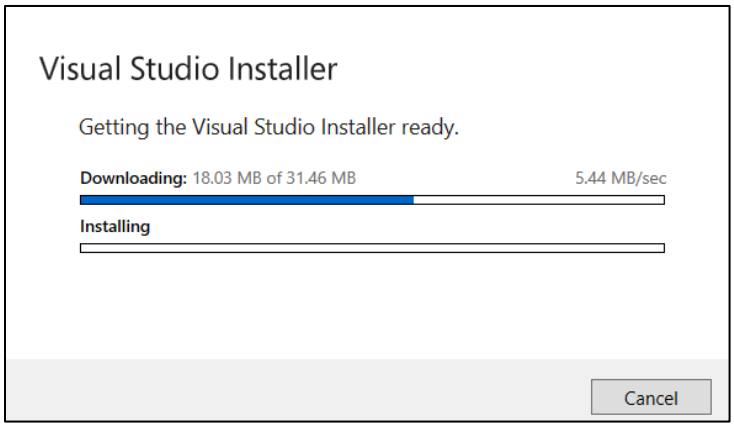
- After setup, double click & press **Continue**.



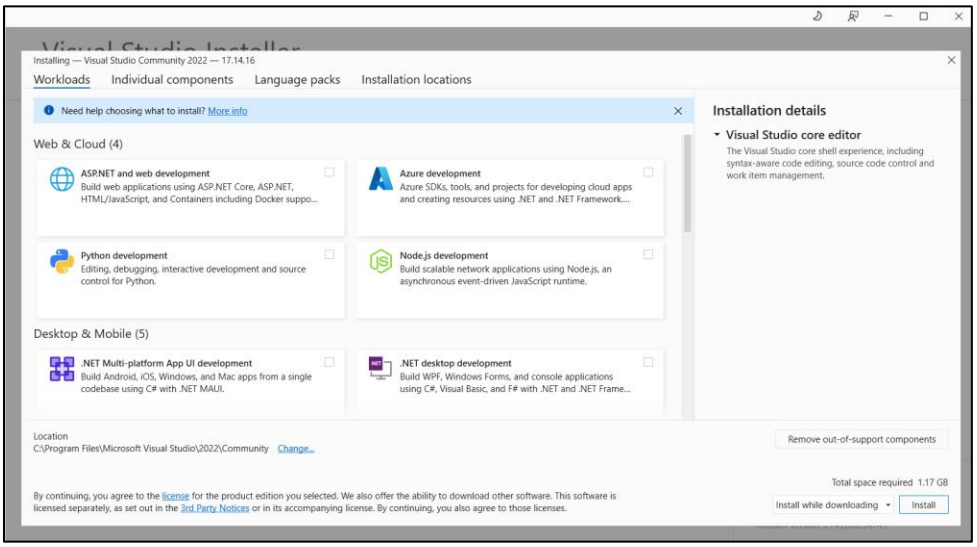
- Start **Installing**.



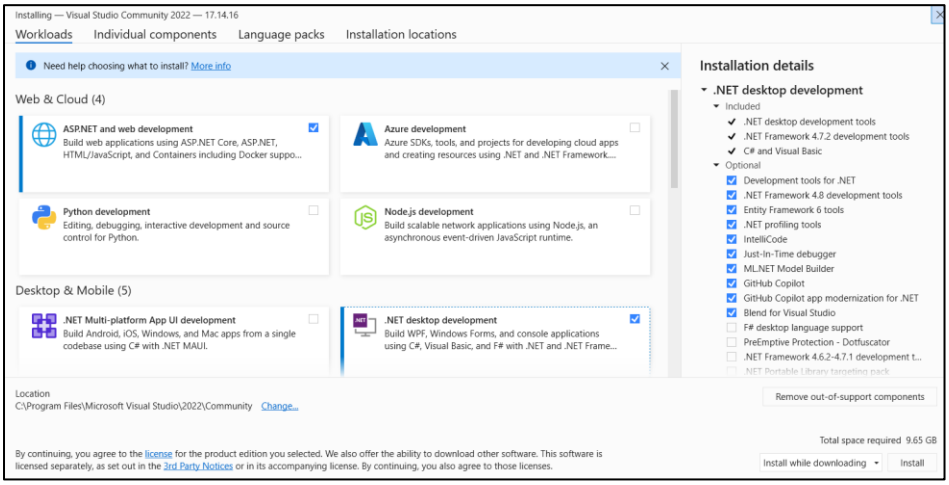
- Wait for Complete **Installation.**



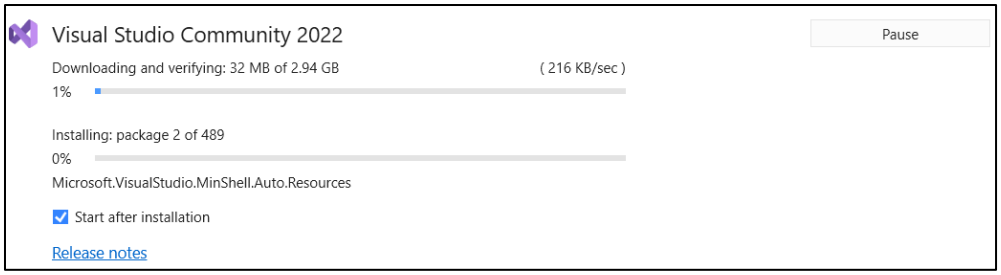
- **Next** this screen will appear.



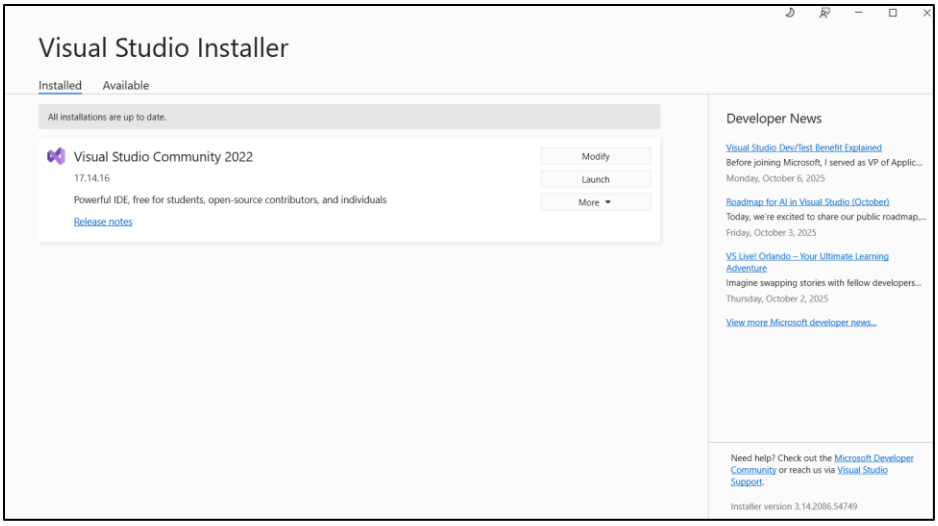
- Choose **ASP.NET and web development** then **.NET desktop development**.
- Now press **Install**.



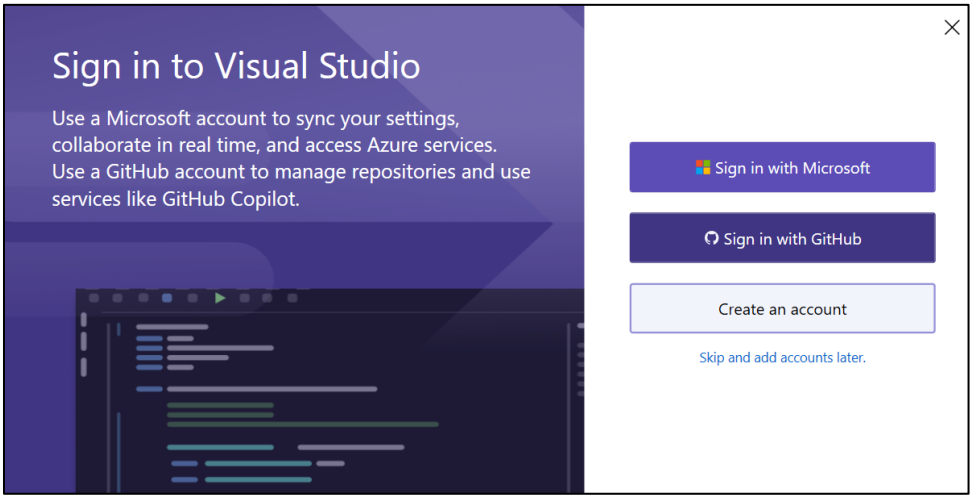
- Start **Installing**.



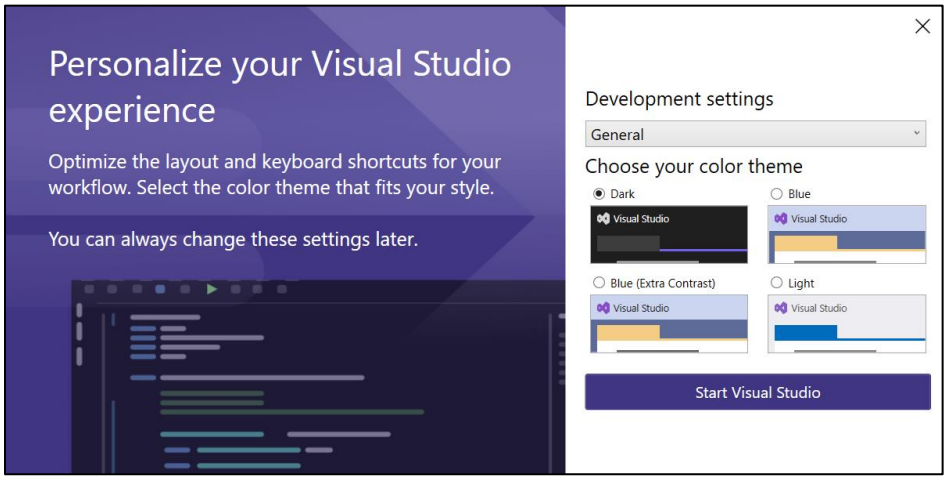
- After Complete **Installation.**



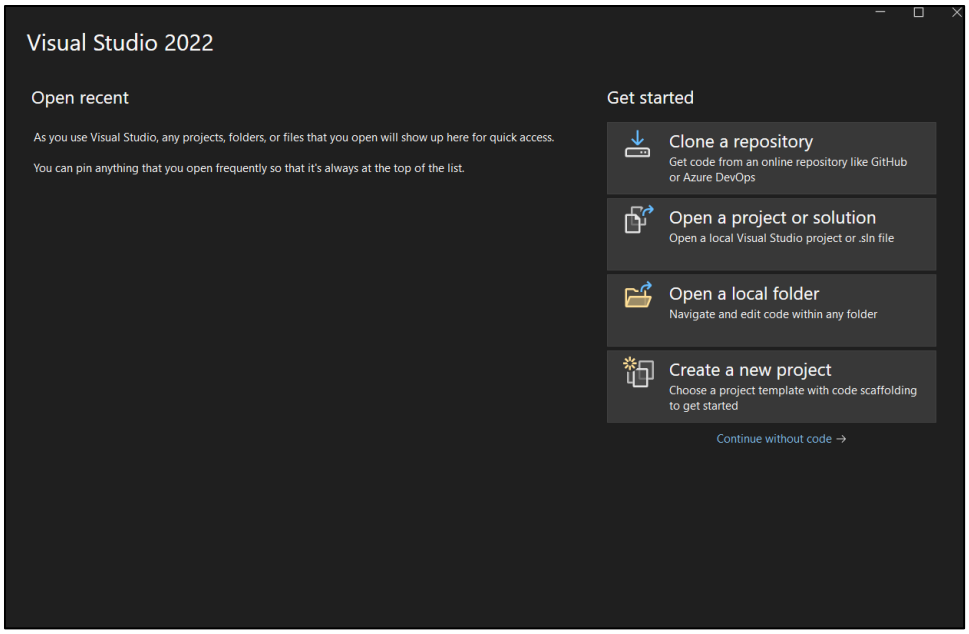
- This screen will show then **click** on **Skip and account later**.



- Then this screen will appear Now **click** on **Start Visual Studio**.



- Then this screen will appear.
- Select **Create a new project** option when we want to code in visual code studio.



LAB 02 & 03

Task 1

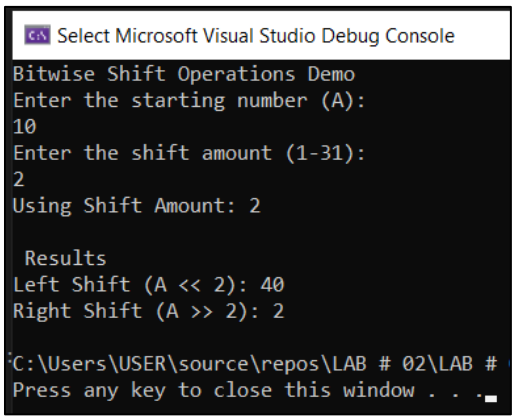
Write a C# program that takes three numbers from the user and performs all arithmetic and shift operations on them.

Program:

```
int numberA;
int shiftAmountInput;
int safeShiftAmount; // Variable used to guarantee a valid shift value

Console.WriteLine("Bitwise Shift Operations Demo");
Console.WriteLine("Enter the starting number (A): ");
numberA = Convert.ToInt32(Console.ReadLine());

Console.WriteLine("Enter the shift amount (1-31): ");
shiftAmountInput = Convert.ToInt32(Console.ReadLine());

// Safety check: Ensures the shift amount is safe for a 32-bit integer.
// If the input is valid (1 to 31), it uses that. Otherwise, it defaults to 2.
safeShiftAmount = (shiftAmountInput > 0 && shiftAmountInput < 32) ? shiftAmountInput : 2;
Console.WriteLine("Using Shift Amount: " + safeShiftAmount);
```

```
// 2. Perform the Shift Operations (Left Shift)
// Left Shift (<<): Moves bits left.
// Result is equivalent to multiplying 'numberA' by 2 raised to the power of 'safeShiftAmount'.
int leftShift = numberA << safeShiftAmount;

// 3. Perform the Shift Operations (Right Shift)

// Right Shift (>>): Moves bits right.
// Result is equivalent to integer dividing 'numberA' by 2 raised to the power of 'safeShiftAmount'.
int rightShift = numberA >> safeShiftAmount;

// 4. Output Section
Console.WriteLine("\n Results");
Console.WriteLine("Left Shift (A << " + safeShiftAmount + "): " + leftShift);
Console.WriteLine("Right Shift (A >> " + safeShiftAmount + "): " + rightShift);
Console.ReadKey();//Control Execution
```

Output:



Task 2

Write a C# program to print the following pattern using loops:

****

***

**

*

Program:

```
for (int i = 4; i >= 1; i--) {
   // Inner loop prints 'i' asterisks on the same line
   for (int j = 0; j < i; j++){
      Console.Write("*"); }

   // Move to the next line after the row is complete
   Console.WriteLine(); // Added this to create a new line
```

```
}
        Console.ReadKey();//Control program execution
```

Output:

```
Microsoft Visual Studio Debug Console
****
***
**
*

C:\Users\USER\source\repos\LAB # 02\LAB #
Press any key to close this window . . .
```

Task 3

Take three the value of three sides of a triangle and tell if three sides are equal or not.

Program:

```
Console.Write("Enter the length of side 1: ");
double side1 = Convert.ToDouble(Console.ReadLine());
Console.Write("Enter the length of side 2: ");
double side2 = Convert.ToDouble(Console.ReadLine());
Console.Write("Enter the length of side 3: ");
double side3 = Convert.ToDouble(Console.ReadLine());

if (side1 <= 0 || side2 <= 0 || side3 <= 0) {
   Console.WriteLine("Invalid input. Side lengths must be positive numbers."); }
else if (side1 == side2 && side2 == side3) {
   Console.WriteLine("The three sides are equal. This is an equilateral triangle.");
}
else if (side1 == side2 || side2 == side3 || side1 == side3) {
   Console.WriteLine("Two sides are equal. This is an isosceles triangle.");
}
else {
   Console.WriteLine("The three sides are not equal. This is a scalene triangle.");}
   Console.ReadKey();
```

Output:

```
C:\Users\USER\source\repos\Lab 03\Lab 03\bin\Debug\net8.0\Lab 03.exe
Enter the length of side 1: 5
Enter the length of side 2: 4
Enter the length of side 3: 7
The three sides are not equal. This is a scalene triangle.
```

```
Microsoft Visual Studio Debug Console
Enter the length of side 1: 5
Enter the length of side 2: 5
Enter the length of side 3: 5
The three sides are equal. This is an equilateral triangle.

C:\Users\USER\source\repos\Lab 03\Lab 03\bin\Debug\net8.0\Lab 03.exe
Press any key to close this window . . .
```

Task 4

Write a C# program that checks whether a given number is a prime number or not. The program should prompt the user to enter a number, and then it should determine if the number is prime or not. (For loop).

Program:

```
bool prime = true;
int n;
Console.WriteLine("Enter a number:");
n = Convert.ToInt32(Console.ReadLine());
if (n <= 1)
{
   prime = false; }
else {
   for (int i = 2; i <= n / 2; i++)   {
      if (n % i == 0)
      {
         prime = false;  break; }   }
}
if (prime)
   Console.WriteLine(n + " Number is Prime");
else
   Console.WriteLine(n + " Number is not Prime");
        Console.ReadKey();//Control program execution
```

Output:

```
Microsoft Visual Studio Debug Console
Enter a number:
121
121 Number is not Prime

C:\Users\USER\source\repos\LAB # 02\LAB #
Press any key to close this window . . .
```

Task 5

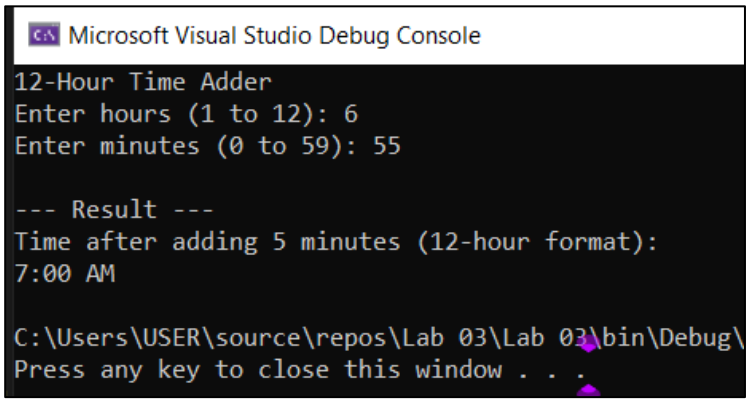
Write a program that will take value of time in minutes and hours from user in 12-hour format. Add 5 minutes and print the time.

Program:

```
int inputHours, inputMinutes;
Console.WriteLine("12-Hour Time Adder ");

// Get Hours (from 1 to 12).
Console.Write("Enter hours (1 to 12): ");
inputHours = Convert.ToInt32(Console.ReadLine());

// Get Minutes (from 0 to 59)
Console.Write("Enter minutes (0 to 59): ");
inputMinutes = Convert.ToInt32(Console.ReadLine());

// --- 3. Time Calculation ---
// Convert input time to total minutes from midnight (e.g., 1:30 PM = 13 hours * 60 + 30)
int initialTotalMinutes = (inputHours * 60) + inputMinutes;

// Add the 5 minutes
int finalTotalMinutes = initialTotalMinutes + 5;

// Calculate the new minutes (0-59) using the remainder operator (%).
int finalMinutes = finalTotalMinutes % 60;

// Calculate the new 24-hour clock hours (0-23), handling day rollover with % 24.
int finalHours24 = (finalTotalMinutes / 60) % 24;

// Convert 24-hour hour (0-23) to 12-hour hour (1-12)
int finalHours12 = finalHours24 % 12;
if (finalHours12 == 0)
{
   // If the remainder is 0 (00:xx or 12:xx), it should display as 12.
   finalHours12 = 12;
}
// Determine AM or PM using the ternary operator (simple if/else).
string period = (finalHours24 >= 12) ? "PM" : "AM";

// Special case: 00:xx (midnight) is always AM.
if (finalHours24 == 0) period = "AM";

// --- 4. Output Section (Manual Formatting) ---

string displayMinutes;

// Custom logic to manually add a leading zero if minutes are < 10.
if (finalMinutes < 10)
{
   displayMinutes = "0" + finalMinutes;
}
else
{
   displayMinutes = finalMinutes.ToString();
}

Console.WriteLine("\n--- Result ---");
Console.WriteLine("Time after adding 5 minutes (12-hour format):");
```

```
// Concatenate the strings directly without String.Format.
Console.WriteLine(finalHours12 + ":" + displayMinutes + " " + period);
        Console.ReadKey();
```

Output:



Task 6

Write the value of date from the user in day/month/year format and calculate his/her age.

Program:

```
int currentDay, currentMonth, currentYear;
int birthDay, birthMonth, birthYear;
int ageInYears;

Console.WriteLine("--- Simple Age Calculator ---");
Console.WriteLine("\nPlease enter the CURRENT date:");
Console.Write("Enter Current Day (DD): ");
currentDay = Convert.ToInt32(Console.ReadLine());
Console.Write("Enter Current Month (MM): ");
currentMonth = Convert.ToInt32(Console.ReadLine());
Console.Write("Enter Current Year (YYYY): ");
currentYear = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("\nPlease enter your BIRTH date:");
Console.Write("Enter Birth Day (DD): ");
birthDay = Convert.ToInt32(Console.ReadLine());
Console.Write("Enter Birth Month (MM): ");
birthMonth = Convert.ToInt32(Console.ReadLine());
Console.Write("Enter Birth Year (YYYY): ");
birthYear = Convert.ToInt32(Console.ReadLine());
//   Age Calculation
// Initial Calculation: Calculate the difference in years.
ageInYears = currentYear - birthYear;
// Check if the person's birthday has passed yet this year, based on the current date provided.
// Scenario 1: If the current month is LESS than the birth month, the birthday hasn't happened.
if (currentMonth < birthMonth)
{
   // Subtract 1 year because the person hasn't had their birthday yet this year.
```

```
    ageInYears = ageInYears - 1;
}
// Scenario 2: If the current month IS the birth month, check the day.
else if (currentMonth == birthMonth)
{
    // If the current day is LESS than the birth day, the birthday hasn't happened.
    if (currentDay < birthDay)  {
        // Subtract 1 year because the person hasn't had their birthday yet this month.
        ageInYears = ageInYears - 1;
    } }
// If the birthday has already passed this year (currentMonth > birthMonth), the initial 'ageInYears' is correct.
Console.WriteLine("\n Result");
Console.WriteLine("Calculating age as of " + currentDay + "/" + currentMonth + "/" + currentYear + ".");
Console.WriteLine("Calculated age is: " + ageInYears + " years.");

Console.ReadKey();
```

Output:

```
Microsoft Visual Studio Debug Console
--- Simple Age Calculator ---

Please enter the CURRENT date:
Enter Current Day (DD): 26
Enter Current Month (MM): 10
Enter Current Year (YYYY): 2025

Please enter your BIRTH date:
Enter Birth Day (DD): 19
Enter Birth Month (MM): 02
Enter Birth Year (YYYY): 2006

 Result
Calculating age as of 26/10/2025.
Calculated age is: 19 years.

C:\Users\USER\source\repos\LAB # 02\LAB # 02\bin\Debug\net8.0\LAB # 02.exe (process 15868)
Press any key to close this window . . .
```

LAB 04

Task 1 (Single D Array)

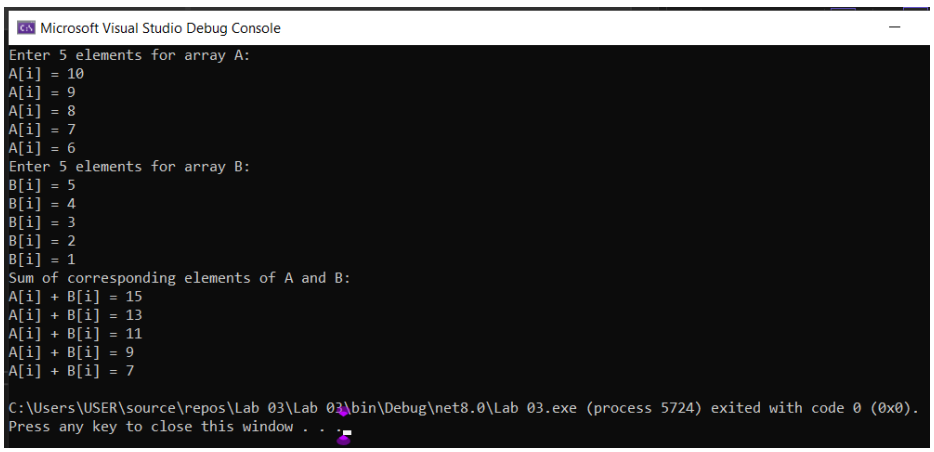
Write a C# program uses three single-dimensional arrays, A, B, and C, each with 5 elements size. It takes input for A and B, adds their corresponding elements, stores the result in C, and displays the sum.

Program:

```
// Get the size of the array from the user
Console.Write("Enter the size of the array: ");
int size = Convert.ToInt32(Console.ReadLine());
// Declare an array of the specified size
int[] array = new int[size];
```

```
// Get the array elements from the user
for (int i = 0; i < size; i++) {
   Console.Write("Enter Elements : ");
   array[i] = Convert.ToInt32(Console.ReadLine()); }
// Count the occurrences of each element
for (int i = 0; i < size; i++) {
   int count = 0;     bool isVisited = false;
   for (int j = 0; j < i; j++) {
      if (array[i] == array[j])    {
         isVisited = true;     break;  } }
   if (!isVisited) {
      for (int j = 0; j < size; j++)    {
         if (array[i] == array[j]) {     count++;      } }
      Console.Write(array[i]+ " Appears ");
      Console.WriteLine(count+ " Times. " );
   } }
Console.ReadKey();
```

Output:



Task 2 (2D Array)

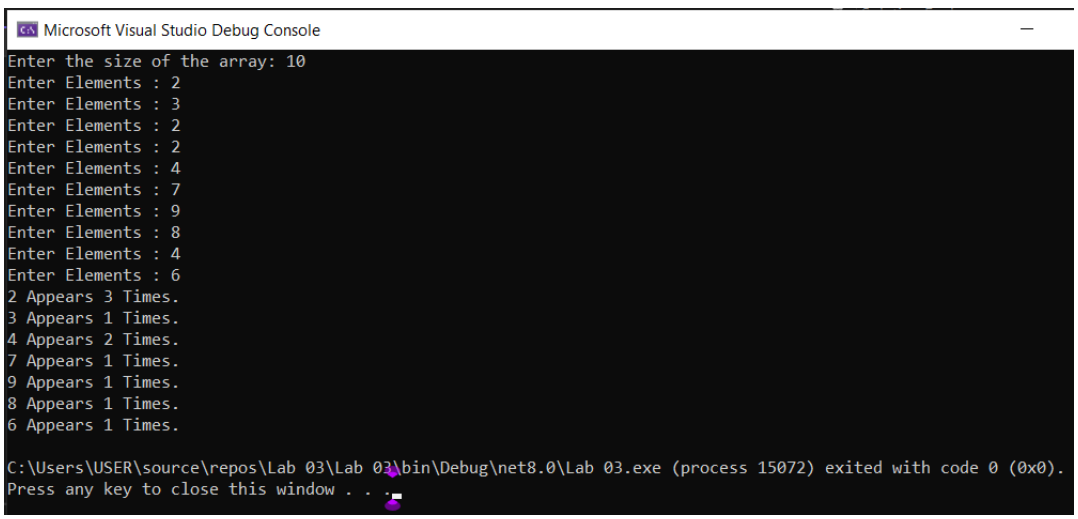
Write a C# console application that counts how many times each element appears in an array. The program should allow the user to input the size and number of elements and then enter the array elements one by one.

Program:

```
// Declare three single-dimensional arrays, A, B, and C, each with 5 elements size
int[] A = new int[5];
   int[] B = new int[5];
   int[] C = new int[5];
   // Take input for A
   Console.WriteLine("Enter 5 elements for array A:");
   for (int i = 0; i < 5; i++) {
      Console.Write("A[i] = ");
      A[i] = Convert.ToInt32(Console.ReadLine()); }
```

```
// Take input for B
Console.WriteLine("Enter 5 elements for array B:");
   for (int i = 0; i < 5; i++) {
      Console.Write("B[i] = ");
      B[i] = Convert.ToInt32(Console.ReadLine()); }
  // Add corresponding elements of A and B, store the result in C
   for (int i = 0; i < 5; i++)
   {
      C[i] = A[i] + B[i]; }
   // Display the sum of arrays
   Console.WriteLine("Sum of corresponding elements of A and B:");
   for (int i = 0; i < 5; i++) {
      Console.WriteLine("A[i] + B[i] = " +C[i]); }
Console.ReadKey();
```

Output:



Task 3 (Jagged Array)

Write a C# program using a jagged array to store the marks of 5 students, where each student has a different number of subjects.
The program should allow the user to enter marks for each subject of every student and then perform the following operations for each student:

1. Calculate the total marks.
2. Calculate the average marks.
3. Find the highest marks.
4. Find the lowest marks.
   Finally, display the results (total, average, highest, and lowest) for all students.

Program:

```
// Declare a jagged array to store the marks of 5 students
// Each student can have a different number of subjects
int[][] studentMarks = new int[5][];
// Loop through each student
for (int i = 0; i < 5; i++) {
   // Ask the user to enter the number of subjects for the current student
   Console.Write("Enter number of subjects for student " + (i + 1) + ": ");
   int subjects = Convert.ToInt32(Console.ReadLine());

   // Initialize the array for the current student with the specified number of subjects
   studentMarks[i] = new int[subjects];

   // Loop through each subject for the current student
   for (int j = 0; j < subjects; j++)
   {
      // Ask the user to enter the marks for the current subject
      Console.Write("Enter marks for subject " + (j + 1) + ": ");
      studentMarks[i][j] = Convert.ToInt32(Console.ReadLine());
   } }
// Loop through each student to calculate and display their results
for (int i = 0; i < 5; i++)
{
   // Initialize variables to store the total, highest, and lowest marks
   int total = 0;
   int highest = studentMarks[i][0];
   int lowest = studentMarks[i][0];
   // Loop through each subject for the current student
   for (int j = 0; j < studentMarks[i].Length; j++)
   {
      // Add the marks for the current subject to the total
      total += studentMarks[i][j];
      // Update the highest marks if the current subject's marks are higher
      if (studentMarks[i][j] > highest)
      {
         highest = studentMarks[i][j];
      }
      // Update the lowest marks if the current subject's marks are lower
      if (studentMarks[i][j] < lowest)
      {
         lowest = studentMarks[i][j];
      } }
   // Calculate the average marks
   double average = (double)total / studentMarks[i].Length;

   // Display the results for the current student
   Console.WriteLine("Student " + (i + 1) + " Results:");
   Console.WriteLine("Total Marks: " + total);
   Console.WriteLine("Average Marks: " + average);
   Console.WriteLine("Highest Marks: " + highest);
   Console.WriteLine("Lowest Marks: " + lowest);
   Console.WriteLine(); // Empty line for better readability
```

```
}
Console.ReadKey(); // Pause the program before exiting
```

Output:

```
Microsoft Visual Studio Debug Console
Enter number of subjects for student 1: 3
Enter marks for subject 1: 59
Enter marks for subject 2: 86
Enter marks for subject 3: 74
Enter number of subjects for student 2: 2
Enter marks for subject 1: 87
Enter marks for subject 2: 92
Enter number of subjects for student 3: 3
Enter marks for subject 1: 76
Enter marks for subject 2: 91
Enter marks for subject 3: 89
Enter number of subjects for student 4: 3
Enter marks for subject 1: 88
Enter marks for subject 2: 92
Enter marks for subject 3: 86
Enter number of subjects for student 5: 2
Enter marks for subject 1: 77
Enter marks for subject 2: 69
```

```
Student 1 Results:
Total Marks: 219
Average Marks: 73
Highest Marks: 86
Lowest Marks: 59

Student 2 Results:
Total Marks: 179
Average Marks: 89.5
Highest Marks: 92
Lowest Marks: 87

Student 3 Results:
Total Marks: 256
Average Marks: 85.33333333333333
Highest Marks: 91
Lowest Marks: 76
```

```
Student 4 Results:
Total Marks: 266
Average Marks: 88.66666666666667
Highest Marks: 92
Lowest Marks: 86

Student 5 Results:
Total Marks: 146
Average Marks: 73
Highest Marks: 77
Lowest Marks: 69


C:\Users\USER\source\repos\Lab 03\Lab 03\bin\Debug\net8.0\
Press any key to close this window . . .
```

Task 4

Write a C# program to store 5 employee names in an array and display them in uppercase using a foreach loop.

```
// Declare an array to store employee names
string[] employeeNames = { "Sana", "Huma", "Esha", "Sofia", "Fatima" };
// Display employee names in uppercase using foreach loop
Console.WriteLine("Employee Names in Uppercase:");
foreach (string name in employeeNames) {
Console.WriteLine(name.ToUpper()); }
Console.ReadKey();
```

Output:

```
Select Microsoft Visual Studio Debug Console
Employee Names in Uppercase:
SANA
HUMA
ESHA
SOFIA
FATIMA

C:\Users\USER\source\repos\Lab 03\Lab 03\bin\Debug\net8.0\Lab 03.exe (process 15620) exited with code 0 (0x0).
Press any key to close this window . . .
```

LAB 05

Task 1

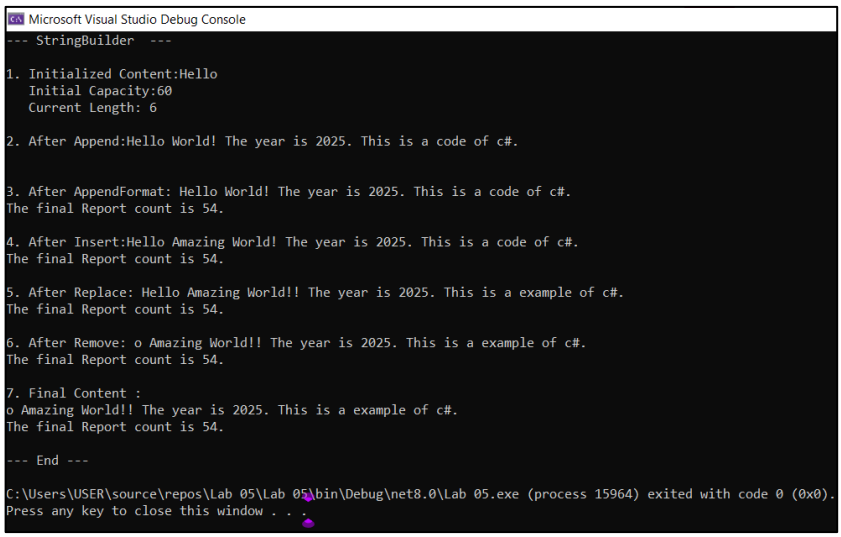
Write a C# program to demonstrate various built-in methods of the **StringBuilder** class, including initializing, appending text, formatting text, inserting, removing, replacing, and converting the StringBuilder content into a string.

Program:

```
using System;
using System.Text; // Required for using the StringBuilder class
    Console.WriteLine("--- StringBuilder  ---");
    // 1. Initialization
    // Capacity is the maximum number of characters the StringBuilder can hold before resizing.
    StringBuilder sb = new StringBuilder("Hello ", 60);
    Console.WriteLine("\n1. Initialized Content:" +sb);
    Console.WriteLine("   Initial Capacity:" +sb.Capacity);
    Console.WriteLine("   Current Length: " +sb.Length);
    // 2. Append (Adding text to the end)
    // Append various data types (strings, numbers, characters).
    sb.Append("World!");
    sb.Append(" The year is ");
    sb.Append(2025);
    sb.AppendLine(". This is a code of c#."); // Appends text followed by a newline
    Console.WriteLine("\n2. After Append:"+sb);
    // 3. AppendFormat (Formatting text with placeholders)
    string item = "Report";
    int total = sb.Length;
    // Use AppendFormat just like String.Format
    sb.AppendFormat("The final {0} count is {1}.", item, total);
    Console.WriteLine("\n3. After AppendFormat: "+sb);
    // 4. Insert (Adding text at a specific index)
    // Insert "Amazing " at index 6 (right after "Hello, ").
    sb.Insert(6, "Amazing ");
    Console.WriteLine("\n4. After Insert:" +sb);
    // 5. Replace (Substituting all occurrences of a string)
    // Replace all instances of "code" with "example".
    sb.Replace("code", "example");
    // Replace a single character '!' with '!!!'
```

```
        sb.Replace('!', '!');
        sb.Replace("!", "!!");
        Console.WriteLine("\n5. After Replace: " +sb);
        // 6. Remove (Deleting characters from a starting index for a specific length)
        // Remove 4 characters starting from index 0 ("Hell").
        sb.Remove(0, 4);
        Console.WriteLine("\n6. After Remove: "+sb);
        // 7. Converting to a String (The final step for output or storage)
// The ToString() method converts the StringBuilder content to an immutable System.String object.
        string finalString = sb.ToString();
        Console.WriteLine("\n7. Final Content :");
        Console.WriteLine(finalString);

            Console.WriteLine("\n--- End ---");
```

Output:



```
Microsoft Visual Studio Debug Console
--- StringBuilder  ---

1. Initialized Content:Hello
   Initial Capacity:60
   Current Length: 6

2. After Append:Hello World! The year is 2025. This is a code of c#.


3. After AppendFormat: Hello World! The year is 2025. This is a code of c#.
The final Report count is 54.

4. After Insert:Hello Amazing World! The year is 2025. This is a code of c#.
The final Report count is 54.

5. After Replace: Hello Amazing World!! The year is 2025. This is a example of c#.
The final Report count is 54.

6. After Remove: o Amazing World!! The year is 2025. This is a example of c#.
The final Report count is 54.

7. Final Content :
o Amazing World!! The year is 2025. This is a example of c#.
The final Report count is 54.

--- End ---

C:\Users\USER\source\repos\Lab 05\Lab 05\bin\Debug\net8.0\Lab 05.exe (process 15964) exited with code 0 (0x0).
Press any key to close this window . . .
```

Task 2

Write a C# program to demonstrate various built-in methods of the **ArrayList** class, including initializing an ArrayList, adding elements, inserting at a specific index, removing elements by value and by index, replacing elements, accessing elements using an index (Get), and displaying all elements along with the total count.one.

Program:

```
using System;
using System.Collections; // Required namespace for ArrayList
        // 1. Initializing an ArrayList
        // An ArrayList can hold elements of any data type.
        ArrayList list = new ArrayList();
        Console.WriteLine("Initialized ArrayList.");

        // 2. Adding Elements (using Add and AddRange)
        list.Add("Makkah");
        list.Add("Madina");
```

```
        list.Add("Masjid-al-Aqsa"); // Can add different data types
        list.Add(true);
        list.Add(100);

// Add a range of items from another array/collection
string[] Fruits = { "Mango", "Orange" };
        list.AddRange(Fruits);
        DisplayList("After adding initial elements", list);

// 3. Inserting an Element at a Specific Index
// Inserts "Masjid-al-Quba" at index 3
list.Insert(3, "Masjid-al-Quba");
        DisplayList("After inserting 'Masjid-al-Quba' at index 3", list);

// 4. Removing Elements
// Remove by value (removes the first occurrence of "Mango")
        list.Remove("Mango");
        DisplayList("After removing 'Mango' by value", list);
/ Remove by index (removes the element at index 5, which is 100)
        list.RemoveAt(5);
        DisplayList("After removing element at index 5", list);

// 5. Replacing Elements
// ArrayList does not have a direct Replace method.
// We simulate replacement by accessing the index directly.
// Change "Orange" to "Apricot"
        list[5] = "Apricot";
        DisplayList("After replacing element at index 5 with 'Apricot'", list);

// 6. Accessing Elements using an Index (Get)
        Console.WriteLine("\n--- Accessing Elements ---");
// Access element at index 1: "Makkah"
        string Dream_Place = list[1].ToString();
        Console.WriteLine("Element at index 1 is: " + Dream_Place);
// Access the first element
        object firstElement = list[0];
        Console.WriteLine("First element is: " + firstElement);
// Example of type checking when accessing:
        if (list[4] is bool){
            Console.WriteLine("Element at index 4 is a boolean value: " + list[4]);
        }
// Helper method to display current list contents and count
void DisplayList(string title, ArrayList currentList)
{
    Console.WriteLine("\n--- " + title + " ---");
    Console.WriteLine("Total Count: " + currentList.Count);
    Console.Write("Elements: ");
    foreach (object item in currentList)
    {
// Formatting output slightly so it's easy to read
        Console.Write("[" + item + "] ");
    }
    Console.WriteLine();
```

}

Output:

```
Select Microsoft Visual Studio Debug Console
Initialized ArrayList.

--- After adding initial elements ---
Total Count: 7
Elements: [Makkah] [Madina] [Masjid-al-Aqsa] [True] [100] [Mango] [Orange]

--- After inserting 'Masjid-al-Quba' at index 3 ---
Total Count: 8
Elements: [Makkah] [Madina] [Masjid-al-Aqsa] [Masjid-al-Quba] [True] [100] [Mango] [Orange]

--- After removing 'Mango' by value ---
Total Count: 7
Elements: [Makkah] [Madina] [Masjid-al-Aqsa] [Masjid-al-Quba] [True] [100] [Orange]

--- After removing element at index 5 ---
Total Count: 6
Elements: [Makkah] [Madina] [Masjid-al-Aqsa] [Masjid-al-Quba] [True] [Orange]

--- After replacing element at index 5 with 'Apricot' ---
Total Count: 6
Elements: [Makkah] [Madina] [Masjid-al-Aqsa] [Masjid-al-Quba] [True] [Apricot]

--- Accessing Elements ---
Element at index 1 is: Madina
First element is: Makkah
Element at index 4 is a boolean value: True

C:\Users\USER\source\repos\Lab 05\Lab 05\bin\Debug\net8.0\Lab 05.exe (process 9792) exited with code 0 (0x0).
Press any key to close this window . . .
```

Task 3

Write a C# program that uses the **StringBuilder** class to create a short story. The program should take four inputs from the user as per wish. Then it should build a story using those inputs and display it on the console.

Program:

```
using System.Text; // Required for StringBuilder
// 1. Take keywords from the user
Console.Write("Enter a character's Name: ");
string name = Console.ReadLine();

Console.Write("Enter a Place: ");
string place = Console.ReadLine();

Console.Write("Enter Emotion: ");
string emotion = Console.ReadLine();
// 2. Use StringBuilder to generate the story
StringBuilder s = new StringBuilder();
// Use AppendLine() to add a line and a newline character at the end
s.AppendLine("\n--- The User's Story ---");
```

```
s.AppendLine("\nOnce upon a time, there was a person named "+name+".");
s.AppendLine("They lived in a beautiful, charming "+place+ ".");
// Use Append() for parts of a sentence, then AppendLine() to finish the line
s.Append("One day, ");
s.Append(name);
s.Append(" decided to go for Peace to ");
s.Append("Makkah & Madina");
s.AppendLine("."); // Finish the sentence with a period and newline
s.Append("They felt ");
s.Append(emotion);
s.Append(" and had a wonderful time!");
s.AppendLine(); // Add an extra newline for spacing

// 3. Display the final story
Console.WriteLine(s.ToString()); // Convert StringBuilder to string for final output
```

Output:

```
Microsoft Visual Studio Debug Console
Enter a character's Name: Eman Nisar Ahmad
Enter a Place: Pakistan
Enter Emotion: Heart-Warming

--- The User's Story ---

Once upon a time, there was a person named Eman Nisar Ahmad.
They lived in a beautiful, charming Pakistan.
One day, Eman Nisar Ahmad decided to go for Peace to Makkah & Madina.
They felt Heart-Warming and had a wonderful time!


C:\Users\USER\source\repos\Lab 05\Lab 05\bin\Debug\net8.0\Lab 05.exe (process 13900) exited with code 0 (0x0).
Press any key to close this window . . .
```

LAB 06

Task 1

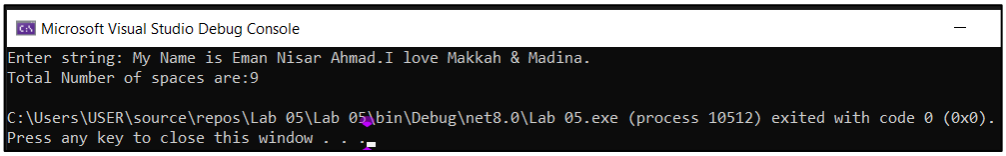
Write a program in C# Sharp to create a function to input a string and count the number of spaces within the string.

Program:

```
class Count
{
  static int count_space(string str) {
      int countspace = 0;
      for (int i = 0; i < str.Length; i++) {
         if (str[i] == ' ') {
            countspace++; }   }
      return countspace;   }
   static void Main()
{
      Console.Write("Enter string: ");
      string str = Console.ReadLine();
```

```
        int spaces = count_space(str);// function call
        Console.WriteLine("Total Number of spaces are:" + spaces);
    } }
```

Output:



Task 2

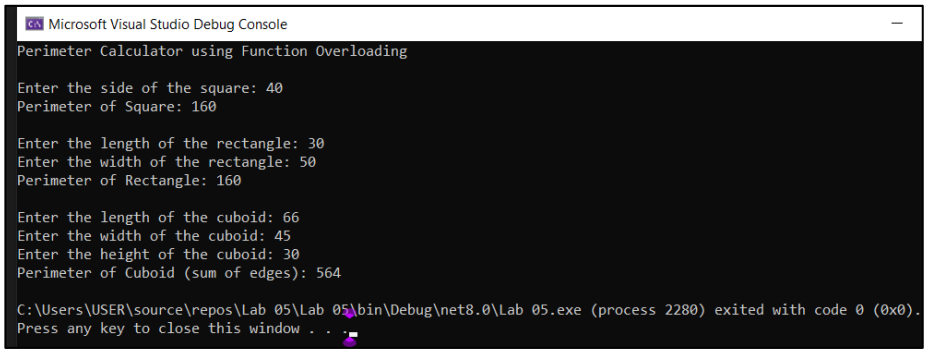
Write a C# program using Function Overloading to calculate the Perimeter of different shapes: Square, Rectangle, Cuboid. Use the same method name Perimeter() but with different parameter lists to perform each calculation.

Program:

```
using System;

class Program {
    // Perimeter of a Square (4 sides)
    static double Perimeter(double side)
    {
        return 4 * side; }
    // Perimeter of a Rectangle (2*(length + width))
    static double Perimeter(double length, double width) {
        return 2 * (length + width); }
    // Perimeter of a Cuboid (Sum of edges: 4*(length + width + height))
    static double Perimeter(double length, double width, double height)
    {
        return 4 * (length + width + height); }
    static void Main()
    {
        Console.WriteLine("Perimeter Calculator using Function Overloading\n");
        // Square
        Console.Write("Enter the side of the square: ");
        double side = double.Parse(Console.ReadLine());
        Console.WriteLine("Perimeter of Square: " + Perimeter(side));
        // Rectangle
        Console.Write("\nEnter the length of the rectangle: ");
        double length = double.Parse(Console.ReadLine());
        Console.Write("Enter the width of the rectangle: ");
        double width = double.Parse(Console.ReadLine());
        Console.WriteLine("Perimeter of Rectangle: " + Perimeter(length, width));
        // Cuboid
        Console.Write("\nEnter the length of the cuboid: ");
        double cuboidLength = double.Parse(Console.ReadLine());
        Console.Write("Enter the width of the cuboid: ");
        double cuboidWidth = double.Parse(Console.ReadLine());
```

```
        Console.Write("Enter the height of the cuboid: ");
        double cuboidHeight = double.Parse(Console.ReadLine());
        Console.WriteLine("Perimeter of Cuboid (sum of edges): " + Perimeter(cuboidLength, cuboidWidth,
cuboidHeight));
    }
}
```

Output:



Task 3

Write a C# program that takes the names and marks of 10 students, stores them, and then sorts the list in two different ways:

1. By **Name** (alphabetically)
2. By **Marks** (in ascending order)

Use two separate functions for each type of sorting and display the sorted lists.

Program:

```
using System;
class Program {
    // Function to sort by name
    static void SortByName(string[] names, int[] marks) {
        for (int i = 0; i < names.Length - 1; i++) {
            for (int j = i + 1; j < names.Length; j++) {
                if (names[i].CompareTo(names[j]) > 0){
                    // swap names
                    string tempName = names[i];
                    names[i] = names[j];
                    names[j] = tempName;
                    // swap marks
                    int tempMark = marks[i];
                    marks[i] = marks[j];
                    marks[j] = tempMark;
                } } }
        Console.WriteLine("\nStudents Sorted by Name:");
        for (int i = 0; i < names.Length; i++)
        {
```

```
        Console.WriteLine(names[i] + " - " + marks[i]);    }   }
// Function to sort by marks
static void SortByMarks(string[] names, int[] marks) {
    for (int i = 0; i < marks.Length - 1; i++)  {
        for (int j = i + 1; j < marks.Length; j++)
        {
            if (marks[i] > marks[j])
            {
                // swap marks
                int tempMark = marks[i];
                marks[i] = marks[j];
                marks[j] = tempMark;
                // swap names
                string tempName = names[i];
                names[i] = names[j];
                names[j] = tempName;
            }  }  }
    Console.WriteLine("\nStudents Sorted by Marks:");
    for (int i = 0; i < names.Length; i++)
    {
        Console.WriteLine(names[i] + " - " + marks[i]);       }  }
static void Main() {
    string[] names = new string[10];
    int[] marks = new int[10];
    Console.WriteLine("Enter names and marks of 10 students:");
    for (int i = 0; i < 10; i++)
    {
        Console.Write("Enter name of student " + (i + 1) + ": ");
        names[i] = Console.ReadLine();
        Console.Write("Enter marks of " + names[i] + ": ");
        marks[i] = int.Parse(Console.ReadLine());
    }
    SortByName(names, marks);   // Sort alphabetically
    SortByMarks(names, marks);  // Sort by marks
} }
```

Output:

```
Microsoft Visual Studio Debug Console
Enter names and marks of 10 students:
Enter name of student 1: Sehrish
Enter marks of Sehrish: 88
Enter name of student 2: Bakhtawar
Enter marks of Bakhtawar: 89
Enter name of student 3: Tahira
Enter marks of Tahira: 78
Enter name of student 4: Huma
Enter marks of Huma: 77
Enter name of student 5: Zobia
Enter marks of Zobia: 69
Enter name of student 6: Mirha
Enter marks of Mirha: 58
Enter name of student 7: Saima
Enter marks of Saima: 76
Enter name of student 8: Kinza
Enter marks of Kinza: 55
Enter name of student 9: Aqsa
Enter marks of Aqsa: 63
Enter name of student 10: Arisha
Enter marks of Arisha: 81
```

```
Students Sorted by Name:
Aqsa - 63
Arisha - 81
Bakhtawar - 89
Huma - 77
Kinza - 55
Mirha - 58
Saima - 76
Sehrish - 88
Tahira - 78
Zobia - 69

Students Sorted by Marks:
Kinza - 55
Mirha - 58
Aqsa - 63
Zobia - 69
Saima - 76
Huma - 77
Tahira - 78
Arisha - 81
Sehrish - 88
Bakhtawar - 89

C:\Users\USER\source\repos\Lab 05\
```

Task 4

Write a C# program to demonstrate the Copy Constructor concept using a class Book.

Each book has a title, author, price, and a Publisher (as another class). The copy constructor should create a deep copy of the book, meaning that if we change the publisher in the copied object, the original one remains unchanged.

Program:

```
using System;
using System.Diagnostics;
using System.Xml.Linq;
class publisher {
   public string Name;
   public string City;
   public publisher(string name, string city) {
      Name = name;
      City = city; }
   public void Display()     {
      Console.WriteLine("Publisher :" + Name);
      Console.WriteLine("Publisher city:" + City);   } }
class Book {
   public String Title;
   public String Author;
   public float Price;
   public publisher p;
   public Book(string title, string author, float price, publisher p)
   {
      Title = title; Author = author;
      Price = price;  this.p = p;  }
```

```
public void display()   {
    Console.WriteLine("\n   Book Details");
    Console.WriteLine("Book Title is: " + Title);
    Console.WriteLine("Book Author is: " + Author);
    Console.WriteLine("Book price is: " + Price);
    Console.WriteLine("Publisher : " + p.Name);
    Console.WriteLine("Publisher city: " + p.City);
}
static void Main() {
    publisher p = new publisher("Oxford", "Lahore");
    Book book = new Book(" C# ", "ABC", 2000, p);
    p.Display();
    book.display();
}  }
```

Output:

Task 5

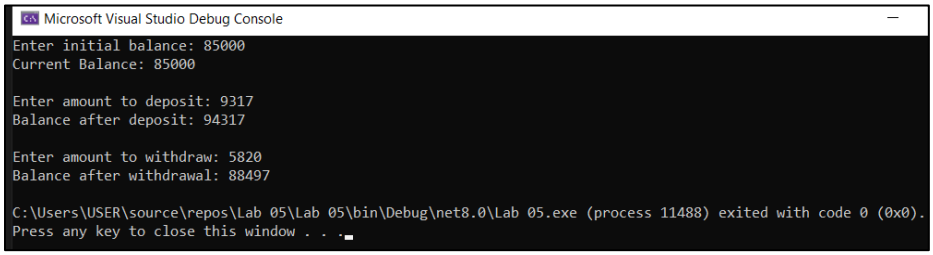
Write a C# program & Create a class BankAccount that hides account balance and allows deposit and withdrawal through properties. Prevent negative balance using encapsulation.

Program:

```
using System;
class BankAccount {
// Private field - hidden from outside
    private double balance;
    // Constructor to initialize balance
    public BankAccount(double initialBalance) {
        if (initialBalance >= 0) balance = initialBalance;
        else
            balance = 0;   }
    // Property to deposit money
    public double Deposit  {   set {
            if (value > 0)   balance += value;
            else
                Console.WriteLine("Deposit amount must be positive!"); }
    }
    // Property to withdraw money
    public double Withdraw {
```

```
        set {  if (value > 0) {
                if (value <= balance)      balance -= value;
                else
                    Console.WriteLine("Insufficient balance! Cannot withdraw more than current balance.");
            }
            else
                Console.WriteLine("Withdrawal amount must be positive!"); }
    }
    // Property to get current balance (read-only)
    public double Balance{  get { return balance; }
    } }
class Program
{
    static void Main()
    {
        Console.Write("Enter initial balance: ");
        double initial = double.Parse(Console.ReadLine());
        BankAccount account = new BankAccount(initial);
        Console.WriteLine("Current Balance: " + account.Balance);
        // Deposit
        Console.Write("\nEnter amount to deposit: ");
        double depositAmount = double.Parse(Console.ReadLine());
        account.Deposit = depositAmount;
        Console.WriteLine("Balance after deposit: " + account.Balance);
        // Withdraw
        Console.Write("\nEnter amount to withdraw: ");
        double withdrawAmount = double.Parse(Console.ReadLine());
        account.Withdraw = withdrawAmount;
        Console.WriteLine("Balance after withdrawal: " + account.Balance);
    } }
```

Output:



LAB 07

Task 1

Write a class **NatPhone** that inherits from the **LocalPhone** class to represent national phone numbers. It includes an additional attribute to

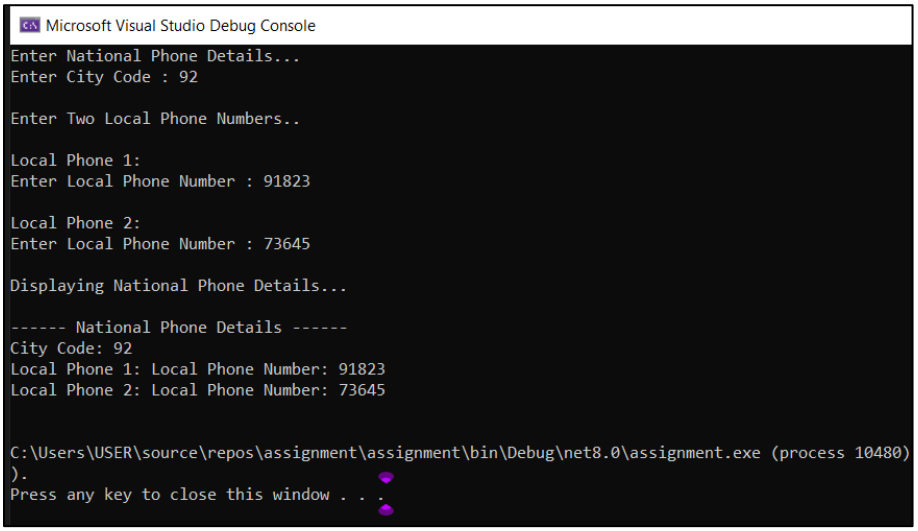
store the city code and an array of two LocalPhone objects to manage multiple local numbers. Member functions are provided to input and display both the city code and the details of the local phones.

Program:

```
using System;
public class LocalPhone
{
   // Variable to store the local phone number
   public long localNumber;
   // Method to input local phone number from user
   public void InputLocal()  {
      Console.Write("Enter Local Phone Number : ");
      // Convert input string to long integer
      localNumber = Convert.ToInt64(Console.ReadLine());
   }
   // Method to display local phone number
   public void DisplayLocal()   {
      Console.WriteLine("Local Phone Number: " + localNumber); }
}
public class NatPhone : LocalPhone
{
   // Variable to store the city code
   public int cityCode;
   // Array to store multiple local phone numbers
   public LocalPhone[] localPhones = new LocalPhone[2];
   // Method to input national phone details from user
   public void InputNatPhone()
   {
      Console.Write("Enter City Code : ");
      // Convert input string to integer
      cityCode = Convert.ToInt32(Console.ReadLine());
      Console.WriteLine("\nEnter Two Local Phone Numbers..");
      for (int i = 0; i < 2; i++)
      {
         // Initialize LocalPhone object for each iteration
         localPhones[i] = new LocalPhone();
         Console.WriteLine($"\nLocal Phone {i + 1}:");
         // Call InputLocal method to get local phone number
         localPhones[i].InputLocal();
      } }
   // Method to display national phone details
   public void DisplayNatPhone()
   {
      Console.WriteLine("\n------ National Phone Details ------");
      Console.WriteLine("City Code: " + cityCode);
      for (int i = 0; i < 2; i++) {
         Console.Write($"Local Phone {i + 1}: ");
         // Call DisplayLocal method to show local phone number
         localPhones[i].DisplayLocal();
      } }
   // Main method to execute the program
```

```
static void Main()
{
    NatPhone np = new NatPhone();
    Console.WriteLine("Enter National Phone Details...");
    np.InputNatPhone();
    Console.WriteLine("\nDisplaying National Phone Details...");
    np.DisplayNatPhone();
    Console.ReadLine();
} }
```

Output:



Task 2

Create a class **BankAccount** with data members account no, account title, and balance, and methods deposit, withdraw, and getaccountdetails. Perform the following tasks:

- Remember no one should be able to access/modify balance directly.
- Inherit class **savingaccount** from bankaccount with additional fields of account type. Override method getaccountdetails.
- Create objects for bankaccount and savingaccount and print account details for both.

Program:

```
using System;
using System.ComponentModel.Design;
using System.Security.AccessControl;
class Bank_Account
{
    private int account_no;
```

```
private string account_title;
private double balance;
public void setdetails(int a, string t, double b)   {
    account_no = a;
    account_title = t;
    balance = b;  }
public double GetBalance()   {
    return balance;
}
public void deposit(double value)  {
    if (value > 0)     {
        balance += value;    }
    else
        Console.WriteLine("deposit amount must be positive");  }
public void Withdraw(double value)  {
    if (value > 0) {
        if (value <= balance)      {
            balance -= value;  }
        else  {
            Console.WriteLine("Insufficient balance! Cannot withdraw more then current balance");
        }  }
    else   {
        Console.WriteLine("Withdrawal amount must be positive"); }    }
 public virtual void getaccountdetails()  {
    Console.Write("Bank account number is:");
    Console.WriteLine(account_no);
    Console.Write("Bank account title is:");
    Console.WriteLine(account_title);
    Console.WriteLine("Remaining balance is:" + balance);  }
class saving_acc : Bank_Account    {
    private string acc_type;
    public void setsavingdetail(int a, string t, double b, string type)   {
        setdetails(a, t, b);      acc_type = type;
    }
    public override void getaccountdetails()     {
        base.getaccountdetails();
        Console.Write("Account type is:");
        Console.WriteLine(acc_type);
    }   }
static void Main()
{
    Bank_Account account = new Bank_Account();
    Console.Write("Enter account number:");
    int accNum = Convert.ToInt32(Console.ReadLine());
    Console.Write("Enter account Title:");
    string accTitle = Console.ReadLine();
    Console.Write("Enter account balance:");
    double accBalance = Convert.ToDouble(Console.ReadLine());
    account.setdetails(accNum, accTitle, accBalance);
    Console.WriteLine("==== Details before deposit ====");
    account.getaccountdetails();
    Console.WriteLine("------------------------------");
    account.deposit(550.12);
```

```
        Console.WriteLine("Details after deposit");
        account.getaccountdetails();
        Console.WriteLine("-----------------------------");
        account.Withdraw(1000.0);
        Console.WriteLine(" Details after withdraw ");
        account.getaccountdetails();
        Console.WriteLine("-----------------------------");
        saving_acc saving = new saving_acc();
        Console.Write("Enter account type for saving account:");
        string type = Console.ReadLine();
        double currentBalance = account.GetBalance();
        saving.setsavingdetail(accNum, accTitle, currentBalance, type);
        Console.WriteLine("\n==== Parent saving details ====");
        account.getaccountdetails();
        Console.WriteLine("\n==== Child saving details ====");
        saving.getaccountdetails();
    } }
```

Output:

```
Microsoft Visual Studio Debug Console
Enter account number:918273645
Enter account Title:Emii
Enter account balance:45000000.43
==== Details before deposit ====
Bank account number is:918273645
Bank account title is:Emii
Remaining balance is:45000000.43
-----------------------------
Details after deposit
Bank account number is:918273645
Bank account title is:Emii
Remaining balance is:45000550.55
-----------------------------
 Details after withdraw
Bank account number is:918273645
Bank account title is:Emii
Remaining balance is:44999550.55
-----------------------------
Enter account type for saving account: Savings

==== Parent saving details ====
Bank account number is:918273645
Bank account title is:Emii
Remaining balance is:44999550.55

==== Child saving details ====
Bank account number is:918273645
Bank account title is:Emii
Remaining balance is:44999550.55
Account type is: Savings

C:\Users\USER\source\repos\assignment\assignment\bin\Debug\net8.0\assignment.exe (process 12828)
```

Task 3

Write a C# Program to rotate an Arraylist of size 10 in left and right Direction three times. Create a Class of ***ArrayRotate*** with data member of integer type arraylist of size 10, property of that member, user define ***displayArray()*** Method, ***RotateLeft(*****ArrayList arl*****)*** Method, ***RotateRight(*****ArrayList arl*****)*** Method.

e.g if the Given

1	2	3	4	5	6	7	8	9	10

ArrayList is:

The after calling ***RotateLeft(int[] arr)*** Method the arraylist would be

4	5	6	7	8	9	10	1	2	3

The after calling ***RotateRight(int[] arr)*** Method the arraylist would be

8	9	10	1	2	3	4	5	6	7

Program:

```
using System;
using System.Collections;

public class ArrayRotate
{
   // Array to store integer values
   public int[] arraylist = new int[10];

   // Method to input array elements from user
   public void InputArray()  {
      Console.WriteLine("Enter 10 integer values:");
      for (int i = 0; i < 10; i++)     {
         arraylist[i] = Convert.ToInt32(Console.ReadLine());
      }   }
   // Method to display array elements
   public void DisplayArray(int[] arr)
   {
      for (int i = 0; i < arr.Length; i++)
      {
         Console.Write(arr[i] + " ");
      }
      Console.WriteLine();
   }
   // Method to rotate array to the left by 3 times
   public void RotateLeft(int[] arr)
   {
      // Rotate the array 3 times
      for (int t = 0; t < 3; t++)        {
         // Store the first element of the array
         int first = arr[0];
         // Shift all elements to the left
         for (int i = 0; i < arr.Length - 1; i++)     {
            // Move the next element to the current position
            arr[i] = arr[i + 1];
         }
         // Place the first element at the end of the array
         arr[arr.Length - 1] = first;
      }    }
   // Method to rotate array to the right by 3 times
   public void RotateRight(int[] arr)    {
      // Rotate the array 3 times
      for (int t = 0; t < 3; t++)
      {
         // Store the last element of the array
```

```
        int last = arr[arr.Length - 1];
        // Shift all elements to the right
        for (int i = arr.Length - 1; i > 0; i--)    {
            // Move the previous element to the current position
            arr[i] = arr[i - 1];
        }
        // Place the last element at the beginning of the array
        arr[0] = last;
    }   }
  public static void Main()
  {
    ArrayRotate obj = new ArrayRotate();
    obj.InputArray();
    Console.Write("\nOriginal Array:");
    obj.DisplayArray(obj.arraylist);
    // Create a copy of the original array
    int[] originalArray = new int[10];
    int index = 0;
    foreach (var item in obj.arraylist)
    {
        originalArray[index++] = item;  }
    // Rotate the original array to the left
    obj.RotateLeft(obj.arraylist);
    Console.Write("Array After RotateLeft 3 times:");
    obj.DisplayArray(obj.arraylist);
    // Rotate the copied array to the right
    obj.RotateRight(originalArray);
    Console.Write("Array After RotateRight 3 times:");
    obj.DisplayArray(originalArray);
    Console.ReadLine();
  }
}
```

Output:

```
Microsoft Visual Studio Debug Console
Enter 10 integer values:
1
2
3
4
5
6
7
8
9
10

Original Array:1 2 3 4 5 6 7 8 9 10
Array After RotateLeft 3 times:4 5 6 7 8 9 10 1 2 3
Array After RotateRight 3 times:8 9 10 1 2 3 4 5 6 7


C:\Users\USER\source\repos\assignment\assignment\bin\Debug\net8.0\
```

Task 4

Write a class Teacher that contains the attributes teacher name, age, and address. It also contains member functions to input and display its attributes. Write a child class Author that inherits from the Teacher class. The Author class contains attributes writer name and number of books written. It also contains member functions to input and display its attributes. Write another child class Scholar that also inherits from the Teacher class. The Scholar class contains attributes research topic and year of research. It also contains member functions to input and display its attributes.

Program:

```
using System;
// Base class representing a Teacher
public class Teacher
{
   // Variables to store teacher details
   public string teacherName;
   public int age;
   public string address;
   // Method to input teacher details from user
   public void InputTeacher()
   {
      Console.Write("Enter Teacher Name: ");
      teacherName = Console.ReadLine();
      Console.Write("Enter Age: ");
      age = Convert.ToInt32(Console.ReadLine());
      Console.Write("Enter Address: ");
      address = Console.ReadLine(); }
   // Method to display teacher details
   public void DisplayTeacher()
   {
      Console.WriteLine("\n--- Teacher Details ---");
      Console.WriteLine("Name: " + teacherName);
      Console.WriteLine("Age: " + age);
      Console.WriteLine("Address: " + address);
   } }
// Derived class representing an Author, inheriting from Teacher
public class Author : Teacher
{
   // Variables specific to Author
   public string writerName;
   public int numberOfBooks;
   // Method to input author details from user
   public void InputAuthor() {
      // Call parent class method to input common teacher details
      InputTeacher();
      Console.Write("Enter Writer Name: ");
      writerName = Console.ReadLine();
```

```
        Console.Write("Enter Number of Books Written: ");
        numberOfBooks = Convert.ToInt32(Console.ReadLine()); }
    // Method to display author details
    public void DisplayAuthor() {
        // Call parent class method to display common teacher details
        DisplayTeacher();
        Console.WriteLine("--- Author Details ---");
        Console.Write("Writer Name: " + writerName);
        Console.Write("Number of Books Written: " + numberOfBooks);
    } }
// Derived class representing a Scholar, inheriting from Teacher
public class Scholar : Teacher {
    // Variables specific to Scholar
    public string researchTopic;
    public int researchYear;
    // Method to input scholar details from user
    public void InputScholar()  {
        // Call parent class method to input common teacher details
        InputTeacher();
        Console.Write("Enter Research Topic: ");
        researchTopic = Console.ReadLine();
        Console.Write("Enter Year of Research: ");
        researchYear = Convert.ToInt32(Console.ReadLine());   }
    // Method to display scholar details
    public void DisplayScholar()  {
        // Call parent class method to display common teacher details
        DisplayTeacher();
        Console.WriteLine("--- Scholar Details ---");
        Console.WriteLine("Research Topic: " + researchTopic);
        Console.WriteLine("Year of Research: " + researchYear);
    }
    // Main method to execute the program
    public static void Main() {
        Console.WriteLine("Enter Author Details...");
        Author author = new Author();
        author.InputAuthor();
        Console.WriteLine("\nEnter Scholar Details...");
        Scholar scholar = new Scholar();
        scholar.InputScholar();
        Console.WriteLine("\n============== OUTPUT ==============");
        author.DisplayAuthor();
        scholar.DisplayScholar();
        Console.ReadLine();
    } }
```

Output:

```
Microsoft Visual Studio Debug Console
Enter Author Details...
Enter Teacher Name: Ma'am Hira
Enter Age: 25
Enter Address: Sialkot, Pakistan
Enter Writer Name: Hira
Enter Number of Books Written: 10

Enter Scholar Details...
Enter Teacher Name: Ma'am Nida
Enter Age: 26
Enter Address: Islamabad, Pakistan
Enter Research Topic: Artificial Intelligence
Enter Year of Research: 2020

============= OUTPUT =============

--- Teacher Details ---
Name: Ma'am Hira
Age: 25
Address: Sialkot, Pakistan
--- Author Details ---
Writer Name: HiraNumber of Books Written: 10
--- Teacher Details ---
Name: Ma'am Nida
Age: 26
Address: Islamabad, Pakistan
--- Scholar Details ---
Research Topic: Artificial Intelligence
Year of Research: 2020


C:\Users\USER\source\repos\assignment\assignment\bin\Debug\net8.0\assignment.exe (process 1440) exited with code 0 (0x0).
Press any key to close this window . . .
```

LAB 08

Create Calculator perform 4 Operations (+, - , * & /)

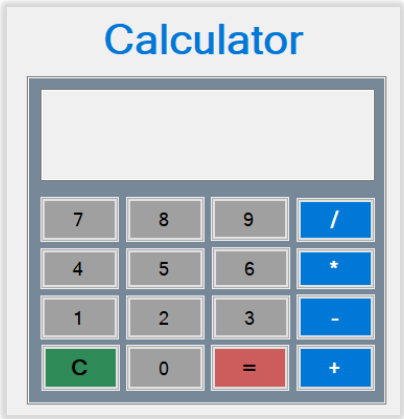
Program:

```
using System;
using System.Collections.Generic;  using System.ComponentModel;
using System.Data;   using System.Drawing; using System.Linq;
using System.Text;  using System.Threading.Tasks;
using System.Windows.Forms;

namespace Calculator
{
   public partial class Form1 : Form
   {
      private double firstnum;
      private string operation = " ";
      private bool isNewnum = true;
      public Form1()  {
         InitializeComponent();  }
      private void button1_Click(object sender, EventArgs e)  {
         if (isNewnum)  {
            textBox1.Text += "1";   isNewnum = false;    }
         else  {
            textBox1.Text += "1";  }
      }
      private void button2_Click(object sender, EventArgs e)       {
         if (isNewnum)  {
            textBox1.Text += "2";       isNewnum = false; }
         else   {
            textBox1.Text += "2";   }
```

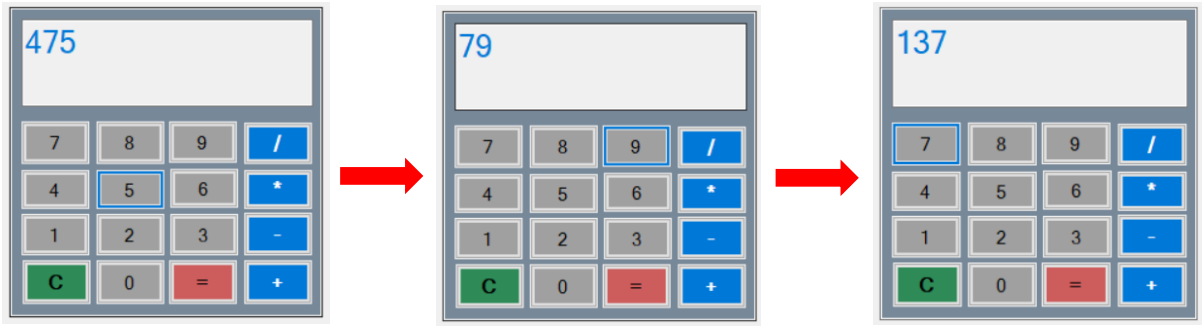
```
    }
    private void button3_Click(object sender, EventArgs e)      {
        if (isNewnum)     {
            textBox1.Text += "3";      isNewnum = false;  }
else   {
            textBox1.Text += "3";   }
    }
    private void button4_Click(object sender, EventArgs e)       {
        if (isNewnum)        {
            textBox1.Text += "4";
            isNewnum = false;      }
        else      {
            textBox1.Text += "4";     }
    }
    private void button5_Click(object sender, EventArgs e)      {
        if (isNewnum)     {
            textBox1.Text += "5";
            isNewnum = false;    }
        else    {
            textBox1.Text += "5";    }
    }
    private void button6_Click(object sender, EventArgs e)      {
        if (isNewnum)      {
            textBox1.Text += "6";
            isNewnum = false;      }
        else     {
            textBox1.Text += "6";        }
    }
    private void button7_Click(object sender, EventArgs e)      {
        if (isNewnum)  {
            textBox1.Text += "7";
            isNewnum = false;  }
        else    {
            textBox1.Text += "7";  }
    }
    private void button8_Click(object sender, EventArgs e)      {
        if (isNewnum)   {
            textBox1.Text += "8";
            isNewnum = false;    }
        else    {
            textBox1.Text += "8";    }
    }
    private void button9_Click(object sender, EventArgs e)      {
        if (isNewnum)     {
            textBox1.Text += "9";
            isNewnum = false;    }
        else   {
            textBox1.Text += "9";   }
    }
    private void button10_Click(object sender, EventArgs e)    {
        if (isNewnum)   {
            textBox1.Text += "0";
            isNewnum = false;   }
        else  {
            textBox1.Text += "0";  }      }
    private void button11_Click(object sender, EventArgs e)   {
        PerformPendingOperation();
        operation = "+";
        isNewnum = true; textBox1.Clear(); }
```

```
        private void button12_Click(object sender, EventArgs e)    {
            PerformPendingOperation();
            operation = "-";
            isNewnum = true; textBox1.Clear(); }
        private void button13_Click(object sender, EventArgs e)   {
            PerformPendingOperation();
            operation = "*";
            isNewnum = true; textBox1.Clear();  }
        private void button14_Click(object sender, EventArgs e)    {
            PerformPendingOperation();
            operation = "/";
            isNewnum = true; textBox1.Clear();  }
        private void PerformPendingOperation()   {
            if (operation != " " && !isNewnum)       {
                double secondnum = Convert.ToDouble(textBox1.Text);
                switch (operation)          {
                    case "+": firstnum += secondnum; break;
                    case "-": firstnum -= secondnum; break;
                    case "*": firstnum *= secondnum; break;
                    case "/": if (secondnum != 0) firstnum /= secondnum; break;         }
                textBox1.Text = firstnum.ToString();     }
            else     {
                firstnum = Convert.ToDouble(textBox1.Text);       }     }
        private void button15_Click(object sender, EventArgs e)      {
            double secondnum = Convert.ToDouble(textBox1.Text);
            double result = 0;
            switch (operation)
            {
                case "+":
                    result = firstnum + secondnum;break;
                case "-":
                    result = firstnum - secondnum; break;
                case "*":
                    result = firstnum * secondnum; break;
                case "/":
                    result = firstnum / secondnum; break;
            }
            MessageBox.Show("Result is: " + result);
            textBox1.Text = result.ToString();
            isNewnum = true;   }
        private void textBox1_TextChanged(object sender, EventArgs e)
        {  }
        private void textBox2_TextChanged(object sender, EventArgs e)
        {  }
        private void button16_Click(object sender, EventArgs e) {
            // Clears all text from the display TextBox
            textBox1.Clear();
        } }
    }
```

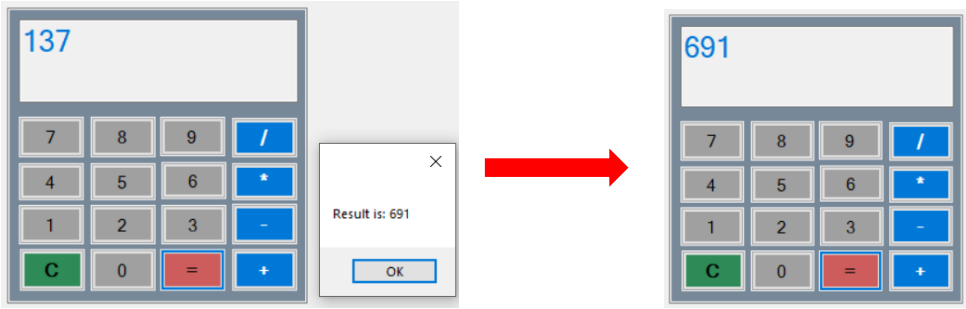
Output:



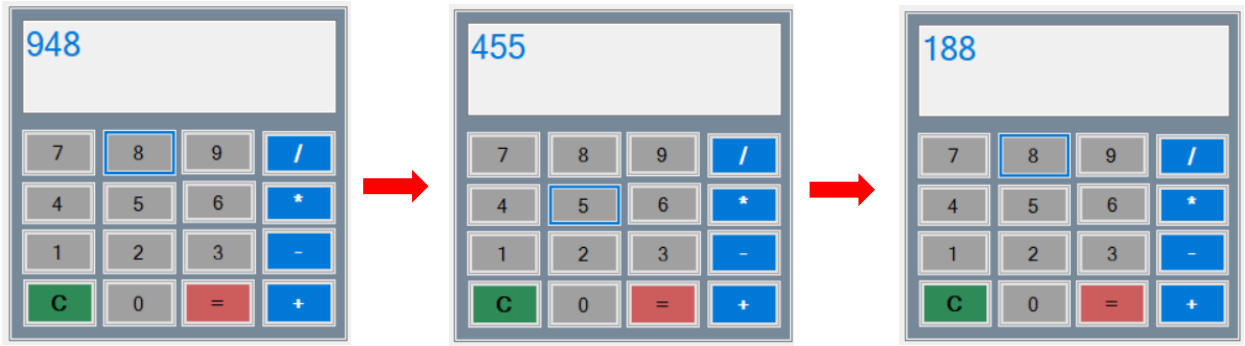
- Now perform ***Addition*** operation. **Enter the first number**: Press the 4, 7, and 5 buttons in sequence. The display will show 475. **Press the addition operator**. Press the + (plus) button. **Enter the second number**: Press the 7 and 9 buttons in sequence. The display will show 79. **Press the addition operator** again. Press the + (plus) button. **Enter the third number**: Press the 1, 3, and 7 buttons in sequence. The display will show 137.



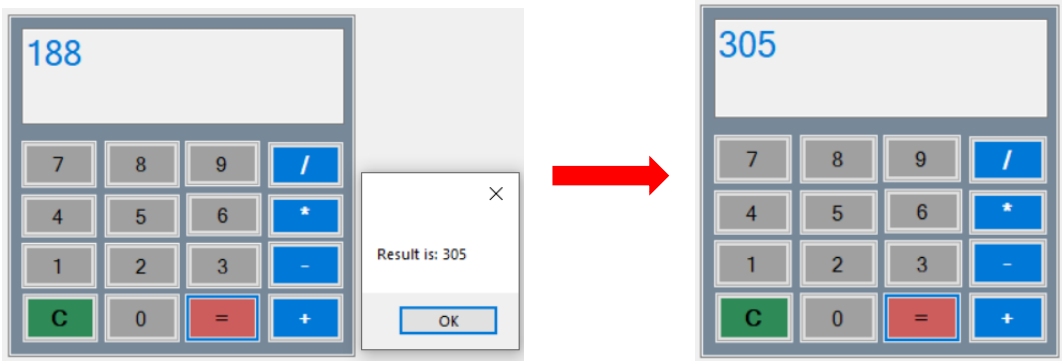
- Output will be shown in text box like ***691***. **Press the equals operator:** Press the = (equals) button to get the final result.



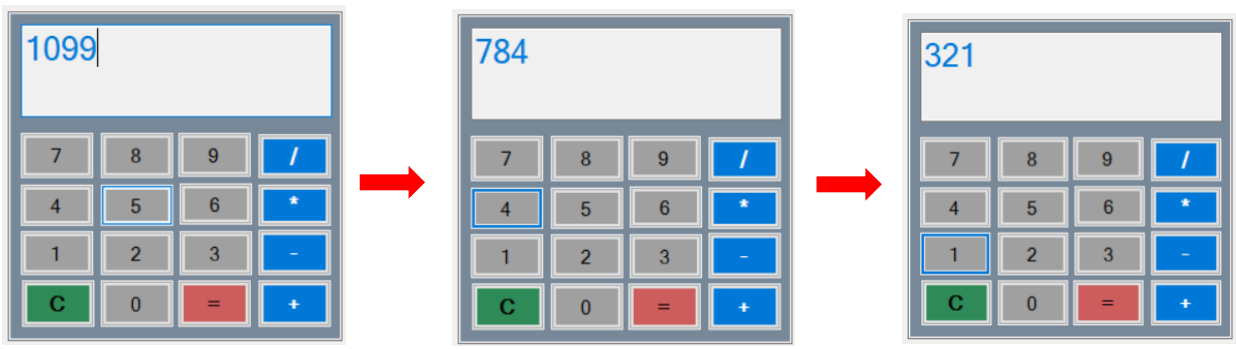
- Now perform ***subtraction*** operation. **Enter the first number:** Press the 9, 4, and 8 buttons in sequence. The display will show 948. **Press the subtraction operator:** Press the - (minus) button. **Enter the second number:** Press the 4, 5, and 5 buttons in sequence. The display will show 455. **Press the subtraction operator again:** Press the - (minus) button. **Enter the third number:** Press the 1, 8, and 8 buttons in sequence. The display will show 188.



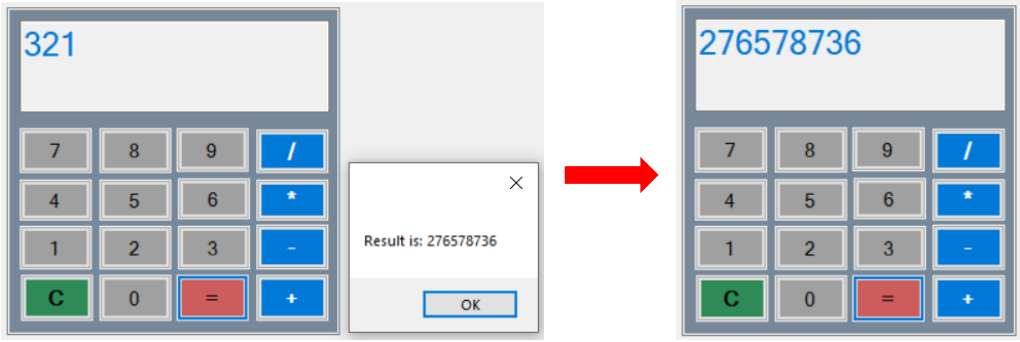
- Output will be shown in text box like ***305***. **Press the equals operator:** Press the = (equals) button to get the final result.



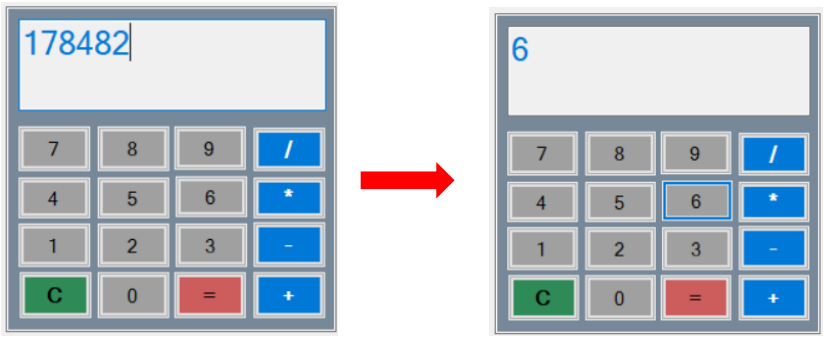
- **Enter the first number:** Press the 1, 0, 9, and 9 buttons in sequence. The display will show 1099. **Press the multiplication operator**. Press the ***** (asterisk) button. **Enter the second number:** Press the 7, 8, and 4 buttons in sequence. The display will show 784. **Press the multiplication operator again**. Press the ***** (asterisk) button. **Enter the third number**. Press the 3, 2, and 1 buttons in sequence. The display will show 321.



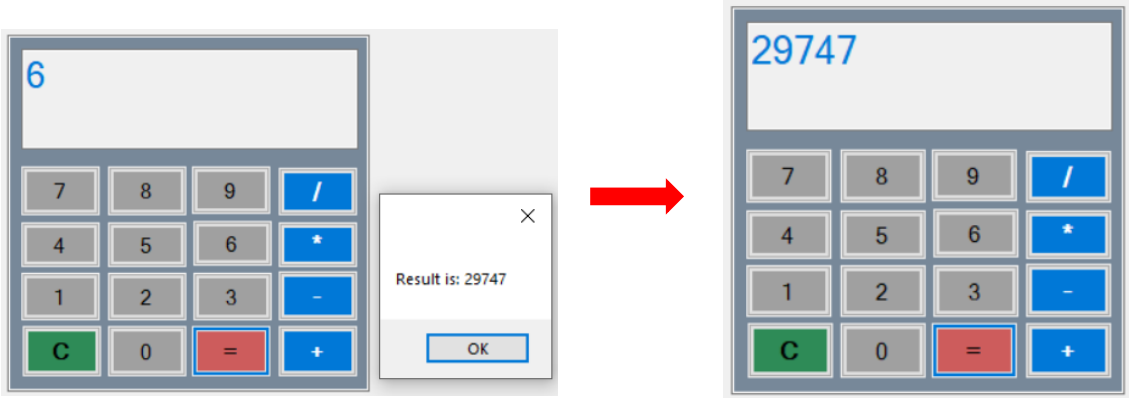
- Press the **equals operator**. Press the = (equals) button to get the final result like **275841696.**



- **Press the digit '1':** Press the **1** button. The display will show 1. **Press the digit '7':** Press the **7** button. The display will show 17. **Press the digit '8':** Press the **8** button. The display will show 178. **Press the digit '4':** Press the **4** button. The display will show 1784. **Press the digit '8' again:** Press the **8** button. The display will show 17848. **Press the digit '2':** Press the **2** button. The display will show 178482.

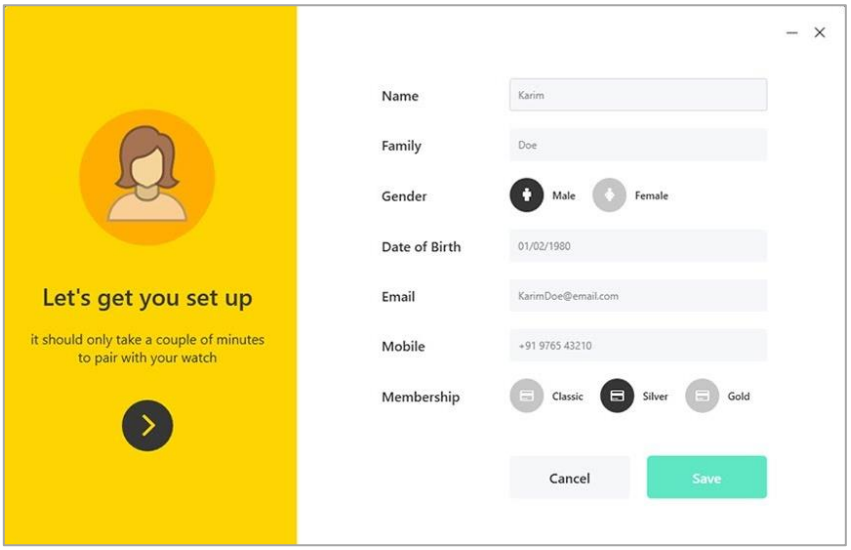


- Press the **equals operator**. Press the = (equals) button to get the final result like **29747.**



LAB 09

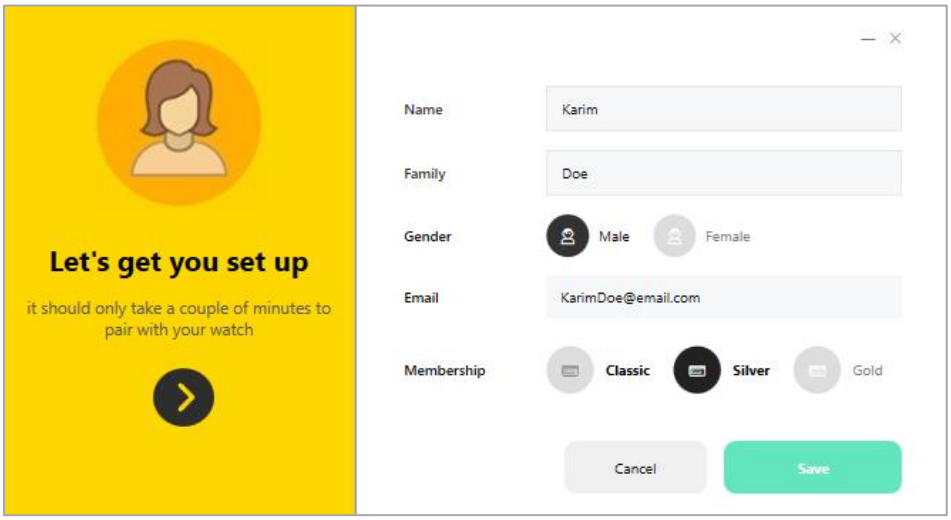
Create the following Form.



Program:

```
<Window x:Class="Form.MainWindow"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
      xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
      xmlns:local="clr-namespace:Form"
      mc:Ignorable="d"
      Title="MainWindow" Height="450" Width="800">
   <Border CornerRadius="20" Background="White" ClipToBounds="True">
      <Grid>
         <Grid.ColumnDefinitions>
            <ColumnDefinition Width="1.5*"/>     <ColumnDefinition Width="2.5*"/>
         </Grid.ColumnDefinitions>
  <Grid Grid.Column="0" Background="#FFD700"></Grid>   <Grid Grid.Column="1" Margin="40,20,40,20">
            <Grid.RowDefinitions>
 <RowDefinition Height="Auto"/>      <RowDefinition Height="*"/>     <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <StackPanel Grid.Row="0" Orientation="Horizontal" HorizontalAlignment="Right" Margin="0,0,0,20">
               <Button Content="—" Background="Transparent" BorderThickness="0" Foreground="Gray" Margin="10,0"/>
               <Button Content="✕" Background="Transparent" BorderThickness="0" Foreground="Gray"/>
            </StackPanel>
            <StackPanel Grid.Row="1" VerticalAlignment="Center">
 <Grid Margin="0,8">
                  <Grid.ColumnDefinitions>
                     <ColumnDefinition Width="120"/>
                     <ColumnDefinition Width="*"/>
                  </Grid.ColumnDefinitions>
                  <TextBlock Text="Name" VerticalAlignment="Center" FontWeight="SemiBold" Foreground="#333"/>
                  <TextBox Grid.Column="1" Text="Karim" Padding="10" Background="#F7F8FA" BorderBrush="#E0E0E0"
BorderThickness="1"/>
               </Grid>
               <Grid Margin="0,8">
                  <Grid.ColumnDefinitions>
                     <ColumnDefinition Width="120"/>
                     <ColumnDefinition Width="*"/>
                  </Grid.ColumnDefinitions>
                  <TextBlock Text="Family" VerticalAlignment="Center" FontWeight="SemiBold" Foreground="#333"/>
                  <TextBox Grid.Column="1" Text="Doe" Padding="10" Background="#F7F8FA" BorderBrush="#E0E0E0"
BorderThickness="1"/>
               </Grid>
               <Grid Margin="0,8">
```

```
                <Grid.ColumnDefinitions>
                  <ColumnDefinition Width="120"/>
                  <ColumnDefinition Width="*"/>
                </Grid.ColumnDefinitions>
                <TextBlock Text="Gender" VerticalAlignment="Center" FontWeight="SemiBold"/>
                <StackPanel Grid.Column="1" Orientation="Horizontal">
                  <StackPanel Orientation="Horizontal" Margin="0,0,20,0">
                    <Border Width="36" Height="36" CornerRadius="18" Background="#333">
                      <TextBlock Text="👤" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center"/>
                    </Border>
                    <TextBlock Text="Male" VerticalAlignment="Center" Margin="8,0,0,0"/>
                  </StackPanel>
                  <StackPanel Orientation="Horizontal">
                    <Border Width="36" Height="36" CornerRadius="18" Background="#DDD">
                      <TextBlock Text="👤" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center"/>
                    </Border>
                    <TextBlock Text="Female" VerticalAlignment="Center" Margin="8,0,0,0" Foreground="Gray"/>
                  </StackPanel>
                </StackPanel>
              </Grid>   <Grid Margin="0,8">
                <Grid.ColumnDefinitions>
               <ColumnDefinition Width="120"/>       <ColumnDefinition Width="*"/>
                </Grid.ColumnDefinitions>
                <TextBlock Text="Email" VerticalAlignment="Center" FontWeight="SemiBold"/>
                <TextBox Grid.Column="1" Text="KarimDoe@email.com" Padding="10" Background="#F7F8FA" BorderThickness="0"/>
              </Grid>
              <Grid Margin="0,15">
                <Grid.ColumnDefinitions>
          <ColumnDefinition Width="120"/>   <ColumnDefinition Width="*"/>
                </Grid.ColumnDefinitions>
                <TextBlock Text="Membership" VerticalAlignment="Center" FontWeight="SemiBold"/>
                <StackPanel Grid.Column="1" Orientation="Horizontal">
                  <Border Width="40" Height="40" CornerRadius="20" Background="#DDD" Margin="0,0,10,0">
                    <TextBlock Text="▬" Foreground="Gray" HorizontalAlignment="Center" VerticalAlignment="Center"/>
                  </Border>
                  <TextBlock Text="Classic" VerticalAlignment="Center" Margin="0,0,20,0" FontWeight="Bold"/>
                  <Border Width="40" Height="40" CornerRadius="20" Background="#222" Margin="0,0,10,0">
                    <TextBlock Text="▬" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center"/>
                  </Border>
                  <TextBlock Text="Silver" VerticalAlignment="Center" Margin="0,0,20,0" FontWeight="Bold"/>
                  <Border Width="40" Height="40" CornerRadius="20" Background="#DDD" Margin="0,0,10,0">
                    <TextBlock Text="▬" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center"/>
                  </Border>
                  <TextBlock Text="Gold" VerticalAlignment="Center" Foreground="Gray"/>
                </StackPanel>          </Grid>       </StackPanel>
            <Grid Grid.Row="2" Margin="0,20,0,0">
              <StackPanel Orientation="Horizontal" HorizontalAlignment="Right">
                <Button Content="Cancel" Width="120" Height="45" Background="#F0F0F0" BorderThickness="0" Margin="0,0,15,0">
                  <Button.Resources>
                    <Style TargetType="Border">
                      <Setter Property="CornerRadius" Value="10"/>    </Style>
                  </Button.Resources>
                </Button>
                <Button Content="Save" Width="150" Height="45" Background="#63E6BE" Foreground="White" FontWeight="Bold"
BorderThickness="0">
                  <Button.Resources>
                    <Style TargetType="Border">
                      <Setter Property="CornerRadius" Value="10"/>         </Style>
                  </Button.Resources>     </Button>
              </StackPanel>       </Grid>   </Grid>
          <Ellipse Width="100" Height="100" HorizontalAlignment="Center" Margin="0,74,0,260">
            <Ellipse.Fill>
              <ImageBrush ImageSource="/pp.png" Stretch="UniformToFill"/>
            </Ellipse.Fill>
          </Ellipse>
          <TextBlock Text="Let's get you set up" FontSize="24" FontWeight="Bold"
                    HorizontalAlignment="Center" TextAlignment="Center" Margin="0,200,0,-288"/>
          <TextBlock Text="it should only take a couple of minutes to pair with your watch" FontSize="14"
                    TextWrapping="Wrap" TextAlignment="Center" Margin="10,247,10,-307" Foreground="#555"/>
```

```
    <Button Background="#2D2D2D" BorderThickness="0" Margin="128,308,120,77">
      <Button.Resources>
        <Style TargetType="Border">
          <Setter Property="CornerRadius" Value="30"/>
        </Style>    </Button.Resources>
      <Path Data="M 2 2 L 12 12 L 2 22"
  Stroke="#FFD700"
  StrokeThickness="4"        StrokeLineJoin="Round"
  StrokeStartLineCap="Round"     StrokeEndLineCap="Round"
  HorizontalAlignment="Center"     VerticalAlignment="Center"
  Margin="5,0,0,0"/>     </Button>
    <Image Margin="78,29,78,260" Source="/Images/Profile.jpeg" Stretch="Fill"/>
</Grid>     </Border>     </Window>
```

Output: